



Security Audit Report

Valdora 18 Decimals Update

v1.0

September 21, 2026

Table of Contents

[Table of Contents](#)

[License](#)

[Disclaimer](#)

[Introduction](#)

[Purpose of This Report](#)

[Codebase Submitted for the Audit](#)

[Methodology](#)

[How to Read This Report](#)

[Code Quality Criteria](#)

[Summary of Findings](#)

[Detailed Findings](#)

- [1. The documented recovery procedure, run during the migration window that the pause flag fails to close, double-scales bonded_amount and settles queued withdrawals at 1% of their value](#)
- [2. Ledgers registered after the v5 migration default to the uzig denom, halting protocol synchronization](#)
- [3. migrate lacks a cw2_guard, so a mistargeted migration irreversibly corrupts protocol accounting](#)
- [4. uzig-denominated precision-loss default prevents withdrawal requests from settling](#)
- [5. The migration rewrites every outstanding withdrawal record in one non-resumable transaction cost scaling with the redemption backlog](#)
- [6. Both contracts store the base denomination under the same key, so recovering from a mis-targeted migration silently leaves the ledger unscaled](#)
- [7. The token registration fee floor and its documentation still express the pre-migration unit](#)

[Proof of Concept](#)

License



THIS WORK IS LICENSED UNDER A [CREATIVE COMMONS ATTRIBUTION-NODERIVATIVES 4.0 INTERNATIONAL LICENSE](https://creativecommons.org/licenses/by-nc/4.0/).

Disclaimer

THE CONTENT OF THIS AUDIT REPORT IS PROVIDED "AS IS", WITHOUT REPRESENTATIONS AND WARRANTIES OF ANY KIND.

THE AUTHOR AND HIS EMPLOYER DISCLAIM ANY LIABILITY FOR DAMAGE ARISING OUT OF, OR IN CONNECTION WITH, THIS AUDIT REPORT.

THIS AUDIT REPORT WAS PREPARED EXCLUSIVELY FOR AND IN THE INTEREST OF THE CLIENT AND SHALL NOT CONSTRUCT ANY LEGAL RELATIONSHIP TOWARDS THIRD PARTIES. IN PARTICULAR, THE AUTHOR AND HIS EMPLOYER UNDERTAKE NO LIABILITY OR RESPONSIBILITY TOWARDS THIRD PARTIES AND PROVIDE NO WARRANTIES REGARDING THE FACTUAL ACCURACY OR COMPLETENESS OF THE AUDIT REPORT.

FOR THE AVOIDANCE OF DOUBT, NOTHING CONTAINED IN THIS AUDIT REPORT SHALL BE CONSTRUED TO IMPOSE ADDITIONAL OBLIGATIONS ON COMPANY, INCLUDING WITHOUT LIMITATION WARRANTIES OR LIABILITIES.

COPYRIGHT OF THIS REPORT REMAINS WITH THE AUTHOR.

This audit has been performed by

Oak Security GmbH

<https://oaksecurity.io>

info@oaksecurity.io

Introduction

Purpose of This Report

Oak Security GmbH has been engaged by Highend Technologies LLC to perform a security audit of the “Valdora 18 Decimals Update” pull request.

The objectives of the audit are as follows:

1. Determine the correct functioning of the protocol, in accordance with the project specification.
2. Determine possible vulnerabilities, which could be exploited by an attacker.
3. Determine smart contract bugs, which might lead to unexpected behavior.
4. Analyze whether best practices have been applied during development.
5. Make recommendations to improve code safety and readability.

This report represents a summary of the findings.

As with any code audit, there is a limit to which vulnerabilities can be found, and unexpected execution paths may still be possible. The author of this report does not guarantee complete coverage (see disclaimer).

Codebase Submitted for the Audit

The audit has been performed on the following target:

Repository	URL	Type	Commit
new-stzig-contracts	https://github.com/Liquid-Zig/new-stzig-contracts	Pull Request #18	Base: new-stzig-contracts@a00c79e720 Head: new-stzig-contracts@0afa20bc38 Compare changes

For repositories reviewed as a Diff or Pull Request above, this audit covers only the changes introduced between the base and head references shown, and does not constitute a full review of the codebase at the head commit.

Scope	<pre>contracts/ ledger/ src/ contract/ exec.rs contract.rs error.rs state.rs staker/ src/ contract/ exec.rs query.rs contract.rs error.rs state.rs</pre>
-------	--

Fixes Verified at Commit

These are the commits at which the client's fixes were reviewed. The status of each finding in this report (Resolved, Partially Resolved or Acknowledged) reflects the code at these commits.

fixes-review: [new-stzig-contracts@60afaab5b3](#)

Note that only fixes to the issues described in this report have been reviewed at this commit. Any further changes such as additional features have not been reviewed.

Methodology

The audit has been performed in the following steps:

1. Gaining an understanding of the code base's intended purpose by reading the available documentation.
2. Automated source code and dependency analysis.
3. Manual line-by-line analysis of the source code for security vulnerabilities and use of best practice guidelines, including but not limited to:
 - a. Race condition analysis
 - b. Under-/overflow issues
 - c. Key management vulnerabilities
4. Report preparation.

How to Read This Report

This report classifies the issues found into the following severity categories:

Severity	Description
Critical	A serious and exploitable vulnerability that can lead to loss of funds, unrecoverable locked funds, or catastrophic denial of service.
Major	A vulnerability or bug that can affect the correct functioning of the system, lead to incorrect states or losses, with medium to high probability.
Minor	A violation of best practices or incorrect usage of primitives, which may not currently have a major impact on security, but may do so in the future or introduce inefficiencies.
Informational	Comments and recommendations of design decisions or potential optimizations, that are not relevant to security. Their application may improve aspects such as user experience or readability. This category may also include opinionated recommendations that the project team might not agree with.

The **status** of an issue can be one of the following: **Resolved**, **Acknowledged**, or **Partially Resolved**.

Note that audits are an important step to improving the security of smart contracts and can find many issues. However, auditing complex codebases has its limits and a remaining risk is present (see disclaimer).

Users of the system should exercise caution. In order to help with the evaluation of the remaining risk, we provide a measure of the following key indicators: **code complexity**, **code readability**, **level of documentation**, and **test coverage**. We include a table with these criteria below.

Note that high complexity or low test coverage does not necessarily equate to a higher risk, although certain bugs are more easily detected in unit testing than in a security audit and vice versa.

Code Quality Criteria

The auditor team assesses the codebase's code quality criteria as follows:

Criteria	Status	Comment
Code complexity	Medium-High	–
Code readability and clarity	Medium	Identifiers still named for retired denom
Level of documentation	High	Thorough plan; runbook contradicts pause code
Test coverage	Medium-High	91% lines; migration window untested

Summary of Findings

No	Description	Severity	Status
1	The documented recovery procedure, run during the migration window that the pause flag fails to close, double-scales bonded_amount and settles queued withdrawals at 1% of their value	Critical	Resolved
2	Ledgers registered after the v5 migration default to the uzig denom, halting protocol synchronization	Major	Resolved
3	migrate lacks a cw2 guard, so a mistargeted migration irreversibly corrupts protocol accounting	Major	Resolved
4	uzig-denominated precision-loss default prevents withdrawal requests from settling	Minor	Resolved
5	The migration rewrites every outstanding withdrawal record in one non-resumable transaction cost scaling with the redemption backlog	Minor	Acknowledged
6	Both contracts store the base denomination under the same key, so recovering from a mis-targeted migration silently leaves the ledger unscaled	Minor	Resolved
7	The token registration fee floor and its documentation still express the pre-migration unit	Informational	Resolved

Detailed Findings

1. The documented recovery procedure, run during the migration window that the pause flag fails to close, double-scales bonded_amount and settles queued withdrawals at 1% of their value

Severity: Critical

- [contracts/staker/src/contract.rs:107](#)
- [contracts/staker/src/contract.rs:114](#)
- [contracts/staker/src/contract.rs:115](#)
- [contracts/staker/src/contract.rs:189-218](#)
- [contracts/ledger/src/contract/exec.rs:514](#)
- [contracts/ledger/src/contract/exec.rs:616](#)
- [contracts/ledger/src/contract.rs:104](#)
- [contracts/ledger/src/contract.rs:120](#)
- [contracts/staker/src/contract/exec.rs:1792-1802](#)
- [contracts/staker/src/contract/exec.rs:2135](#)
- [contracts/staker/src/contract/exec.rs:2254-2256](#)
- [contracts/staker/src/contract/exec.rs:2347-2353](#)
- [contracts/staker/src/contract/exec.rs:3285](#)
- [contracts/staker/src/contract/exec.rs:3300-3306](#)
- [contracts/staker/src/contract/exec.rs:3352-3366](#)

The migration runs as two separate transactions. The staker contract is migrated by a chain-level `MsgMigrateContract` from the WebAssembly admin. The ledgers are migrated by `ExecMsg::MigrateLedger`, which is gated on the internal `CONTRACT_OWNER`. Section 13 of the migration plan records that these are two different keys.

The runbook fences the interval between the two transactions with the pause flag. Section 3A control 5 requires the staker to be "paused for the whole window". Section 5 step 2 states that pause blocks "deposits, redeems, sync, and withdrawals". The code does not implement that control. `PAUSED` is read at exactly nine sites, and none of them gates `ReconcileLedgers`, `DistributeLosses`, `Sync`, `CollectUnbondedAmounts` or `DistributeRewardsAndFees`. The dispatcher checks the flag only for `Deposit`, `Redeem`, `WithdrawUnbonded` and `WithdrawUnbondedById`.

A `ReconcileLedgers` call inside that window causes unrecoverable damage. `execute_reconcile` reads the bonded amount live from the chain in line 514, then persists it in line 616. After the upgrade height that value is already rescaled. `apply_v5 redenomination` therefore multiplies it by 10^{12} a second time in `contracts/ledger/src/contract.rs:120`. Its only guard, in line 104, tests the stored denomination, which is still `uzig` at that point. The next reconciliation reads the

resulting 10^{12} over-count as a near-total slash, and `execute_distribute_losses` writes `FUNDS_RAISED_BALANCE` down in line 3306. The guard in lines 3300-3304 is an underflow check rather than a floor.

The trigger is the project's own documented procedure, not a hypothetical sequence. The scheduled worker cannot reach this window, because `runCron` aborts on a hardcoded "uzig" balance check before its first broadcast, and that check returns a zero coin after the upgrade. That failure is exactly the condition described in `contracts/staker/README.md:315-320`, under the heading "Recovery Procedures: Manual sync execution (if automated sync fails)". Those instructions tell the operator to run `reconcile_ledgers` and then `distribute_losses` by hand. An operator whose automation has just failed, and who follows the project's own recovery instructions, executes the two commands that cause the damage. Nothing in the runbook or in the contracts warns against this.

Consequently the loss is realized rather than merely recorded. Already-queued redemptions settle through `process_withdrawal` at `share_value` scaled by the slash factor. Lines 3352-3366 clamp that factor to a 1% floor, so users with pending withdrawals are paid 1% of what they are owed. The payout is irreversible. That floor is applied 46 lines after `FUNDS_RAISED_BALANCE` has been written, so it does not protect the balance itself. Every remaining share is repriced at $1/10000$ of its backing. Applying the mainnet figure from section 11.1, a `funds_raised` of 200,011,600,442,610 becomes 2.000116×10^{22} azig instead of 2.000116×10^{26} . None of the six write sites for `FUNDS_RAISED_BALANCE` is an administrative setter. The emergency withdrawal is itself capped by the corrupted value, and it then leaves that value at zero. The only escape hatch is therefore disabled by the damage it would repair.

A Sync call in the same window causes a second and milder defect. It reads the balance in line 2135 under the stale denomination and receives zero, because the bank module answers a query for a nonexistent denomination with a zero coin. Line 2254 then clamps `remaining_amount` to that zero, so nothing is bonded. Lines 2347-2353 zero `BUFFER_DEPOSITS` and `BUFFER_REWARDS` regardless. The funds are not lost. They remain counted in `FUNDS_RAISED_BALANCE` and stay available to later withdrawal settlement. They are never delegated, however, so every holder forgoes the staking yield on them, and the buffer accounting becomes silently wrong with no error emitted.

Only `get_bonded_amount` is a live corruption vector in this window. The other chain-reading handlers neutralize themselves, because they filter on the still-uzig stored denomination and receive zeros.

We classify this issue as critical since it pays real users 1% of their entitlement and destroys the backing of every remaining share. No shipped message can restore that state. The trigger is the project's own documented recovery procedure, run by an operator doing exactly what the role requires.

→ [Proof of Concept: P.1](#)

Recommendation

We recommend setting `BONDED_AMOUNT` from the chain in `contracts/ledger/src/contract.rs:120`, using the same `query_delegation` call that `execute_reconcile` already treats as the source of truth in `contracts/ledger/src/contract/exec.rs:616`, rather than multiplying the stored value. This makes the migration independent of when it runs, and it is the minimal patch. It is safe precisely because `BONDED_AMOUNT` is the only migrated item with a live chain counterpart.

We also recommend gating `Sync`, `ReconcileLedgers`, `CollectUnbondedAmounts`, `DistributeRewardsAndFees` and `DistributeLosses` on the pause flag, so that the documented control actually exists. That gate must ship in a separate migration deployed **before** the upgrade height. A gate shipped inside the v5 migration cannot protect the window it exists for, because it only becomes active once that window has already closed.

Adding `execute_reconcile_ledgers` to `PROTOCOL_MODIFIERS` is available immediately and needs no code change, but it is a partial measure rather than a fix. `assert_authorization` returns early for `CONTRACT_OWNER` in `contracts/staker/src/contract/exec.rs:47-50` before it consults the modifier list in lines 52-57, so the list restricts role holders and the scheduled worker's key, and does not restrict the owner. The owner is the actor in the scenario above.

Finally, we recommend that `execute_sync` treat an unreadable balance as an error rather than as zero. Both the runbook and the recovery procedure in `README.md` should state that reconciliation must not run between the chain upgrade and the ledger migration.

Status: Resolved

2. Ledgers registered after the v5 migration default to the uzig denom, halting protocol synchronization

Severity: Major

- [contracts/ledger/src/state.rs:8](#)
- [contracts/ledger/src/state.rs:12-18](#)
- [contracts/ledger/src/contract/init.rs:49-74](#)
- [contracts/staker/src/contract/exec.rs:205-220](#)
- [contracts/staker/src/contract/exec.rs:2291-2298](#)
- [contracts/staker/src/contract/exec.rs:2317-2319](#)
- [contracts/ledger/src/contract/exec.rs:80-81](#)

The `base_denom` helper returns "uzig" whenever the `BASE_DENOM` item is absent, on the assumption that an absent item implies pre-v5 state. The item is written only by `apply_v5 redenomination` in `contracts/ledger/src/contract.rs:138`, which is reachable exclusively through `migrate`. The `instantiate` function initializes every other item but never writes `BASE_DENOM`, and `LedgerInstantiateMsg` in `contracts/staker/src/msg.rs:420-423` provides no field for it.

The assumption does not hold for a ledger created after the migration. The `staker` contract instantiates ledgers at runtime in `execute_register_ledger`, and `BASE_DENOM` is per-instance storage, so the new instance starts with an empty store, is never migrated, and resolves its `denom` to "uzig" while the `staker` contract resolves its own to "azig".

The impact is a protocol-wide denial of service:

- In `contracts/staker/src/contract/exec.rs:2291-2298`, `execute_sync` attaches bonded funds denominated in "azig", and the receiving ledger rejects them because `must_pay` searches for "uzig". The bond messages are attached with `add_messages` in lines 2317-2319 rather than as submessages with an error reply, so the failure aborts the whole transaction. Every subsequent `execute_sync` call reverts for as long as the ledger is registered, which halts deposit bonding, reward harvesting, and withdrawal processing for the entire protocol.
- The affected ledger is inoperable when called directly, since `execute_unbond` in line 178, `execute_redelegate` in line 738, and `execute_withdraw_balance` in line 817 construct a `Coin` from a `denom` that does not exist on the chain after the redenomination.

The same defect applies to a clean deployment of the `staker` contract, whose `instantiate` also leaves `BASE_DENOM` unset, so `must_pay` in `contracts/staker/src/contract/exec.rs:598` rejects every user deposit.

We classify this issue as major since it leads to an incorrect state and a denial of service of the protocol's core synchronization loop, and it is triggered by the intended ledger registration flow rather than by an abuse of privilege.

Recommendation

We recommend propagating the denom from the staker contract to each ledger at instantiation, instead of inferring it from the absence of the item. The staker contract is the only creator of ledgers, since `contracts/staker/src/contract/exec.rs:212` is the sole instantiation site and `LEDGERS.save` in `contracts/staker/src/contract.rs:397` is the sole registration site, and it is migrated and therefore holds the correct denom. It is consequently able to supply the value to every ledger it creates:

1. Add a `base_denom` field to `LedgerInstantiateMsg` in `contracts/staker/src/msg.rs:420-423` and to the ledger's `InstantiateMsg` in `contracts/ledger/src/msg.rs:5-8`. Both structures describe the same payload and must be changed together.
2. Populate the field in `execute_register_ledger` in `contracts/staker/src/contract/exec.rs:205-208` from `base_denom(deps.storage)`, which is the value the staker contract's own migration has already set.
3. Save it in the ledger's `instantiate`, which currently writes every other item but never writes `BASE_DENOM`, by adding `BASE_DENOM.save(deps.storage, &msg.base_denom)?` alongside the other initializations in `contracts/ledger/src/contract/init.rs:49-74`.

The staker contract requires the same fix: it must also save the denom in `instantiate`, in `contracts/staker/src/contract/init.rs:120-200`.

Status: Resolved

3. migrate lacks a cw2 guard, so a mistargeted migration irreversibly corrupts protocol accounting

Severity: Major

- [contracts/staker/src/contract.rs:568-577](#)
- [contracts/ledger/src/contract.rs:142-151](#)
- [contracts/staker/src/contract/exec.rs:2988-3011](#)
- [contracts/staker/src/contract/exec.rs:3059-3097](#)

Both `migrate` functions call `set_contract_version` and then immediately apply `apply_v5 redenomination`, which multiplies every stored base-currency amount by 10^{12} and rewrites `BASE_DENOM`. Neither function first calls `cw2::get_contract_version` to confirm that the

storage it is about to rescale belongs to the contract the code was written for. Only `set_contract_version` is imported, in `contracts/staker/src/contract.rs:7` and `contracts/ledger/src/contract.rs:6`, and it overwrites the stored record unconditionally rather than validating it. The rescale is one-way and has no inverse, and its sole idempotency guard is the `BASE_DENOM` value that the same routine writes, so a single incorrect invocation is unrecoverable.

The migration dispatchers provide no compensating validation. `execute_migrate_token` in lines 2988-3011 and `execute_migrate_ledger` in lines 3059-3097 accept `new_code_id` as a bare u64 and place it directly into `WasmMsg::Migrate` without checking that the code ID corresponds to the contract type being migrated. During the v5 rollout the operator holds three code IDs simultaneously, for the `staker` contract, the `ledger`, and the `token`, and the `staker` contract is the `wasm` admin of all of them, so all three migrations are driven from the same place with the same message shape.

Two consequences follow:

- A transposed code ID migrates a contract into foreign code and is silently accepted. Passing the ledger code ID to `MigrateToken` runs the ledger's `migrate` against the token's storage: `set_contract_version` stamps the record `stzig-ledger` without reading the existing `stzig-token` value, `apply_v5 redenomination` reads an absent `BASE_DENOM` and therefore proceeds as if the contract were pre-v5, and it writes `BASE_DENOM` as "azig" in a storage namespace the item does not belong to. The token contract is left running ledger code with a falsified version record, and nothing in either contract objects. The mirror case, passing the `staker` code ID to `MigrateLedger`, fans the wrong migration out across every registered ledger in a single transaction.
- The redenomination is unconditional with respect to the chain. The guard tests only the stored `denom`, which cannot observe whether the chain itself has redenominated. Because `migrate` is also the entry point for a version bump and for admin rotation, invoking it on a deployment whose chain still uses `uzig`, such as a testnet or a fork, silently inflates `FUNDS_RAISED_BALANCE` and every ledger amount by 10^{12} and sets a `denom` that does not exist there.

In either case the `stZIG` exchange rate, which is derived from `FUNDS_RAISED_BALANCE`, is corrupted by a factor of 10^{12} against a token supply that is deliberately left unscaled, and there is no rollback path.

We classify this issue as major since it leads to irreversible incorrect state across the entire protocol and the missing check is the standard `cw2` guard for exactly the code path this change introduces. Although the trigger is an operator action, the issue does not concern privilege abuse: the operator performs an intended migration and a single mistyped parameter is accepted without objection, which is the class of error the guard exists to prevent.

Recommendation

We recommend calling `cw2::get_contract_version` before any state change in both `migrate` functions, and returning an error unless the stored contract field matches the expected name, "stzig-staker" in `contracts/staker/src/contract.rs:568-577` and "stzig-ledger" in `contracts/ledger/src/contract.rs:142-151`, and unless the stored version is an accepted upgrade source. Only then should `set_contract_version` and `apply_v5 redenomination` run.

We further recommend deriving the redenomination guard from that version rather than from `BASE_DENOM`, so that the condition does not depend on a value the routine itself writes, and validating `new_code_id` in `execute_migrate_token` and `execute_migrate_ledger` against the target contract's `cw2` record before dispatching `WasmMsg::Migrate`.

Status: Resolved

4. uzig-denominated precision-loss default prevents withdrawal requests from settling

Severity: Minor

- [contracts/staker/src/state.rs:28](#)
- [contracts/staker/src/state.rs:31](#)
- [contracts/staker/src/state.rs:35-41](#)
- [contracts/staker/src/contract.rs:519-520](#)
- [contracts/staker/src/contract/init.rs:120-200](#)
- [contracts/staker/src/contract/exec.rs:1809](#)
- [contracts/staker/src/contract/exec.rs:1818](#)

The `MAX_PRECISION_LOSS` item holds the rounding tolerance applied when a withdrawal request is settled against the available funds. It is written only by `apply_v5 redenomination` in `contracts/staker/src/contract.rs:519-520`, which is reachable exclusively through `migrate`, and the `max_precision_loss` helper in `contracts/staker/src/state.rs:35-41` falls back to `MAX_PRECISION_LOSS_DEFAULT` whenever the item is absent. That constant is 6000 in line 31, a value denominated in uzig. The `initiate` function in `contracts/staker/src/contract/init.rs:120-200` initializes the contract's other configuration but never writes `MAX_PRECISION_LOSS`.

The fallback therefore assumes that an absent item implies pre-v5 state, which does not hold for a staker contract deployed after the migration. Such an instance applies a uzig-scale threshold to azig-scale amounts, a tolerance 10^{12} times smaller than intended and effectively zero.

The value is consumed in `contracts/staker/src/contract/exec.rs:1818`, where the condition `adjusted_share_value.le(&remaining_funds)` || `precision_loss.le(&max_precision_loss(deps.storage))` determines whether a request is settled. When the condition does not hold, the request is skipped and left in `WITHDRAWAL_QUEUE`, so a request whose adjusted share value exceeds the remaining funds by a rounding amount is never settled and remains queued until further funds arrive, rather than being paid out from the remaining balance as intended. No administrative setter exists for the item, so the tolerance cannot be corrected without a further migration, and the comment in line 1809 stating that precision loss is allowed "up to 6000 uzig" no longer describes the post-migration configuration.

This issue shares a root cause with the finding reported in issue #1 .

We classify this issue as minor since it does not lead to a loss of funds and affects only newly deployed instances, but it produces an incorrect configuration that silently disables a settlement path intended to be available.

Recommendation

We recommend writing `MAX_PRECISION_LOSS` in the staker contract's `initiate` in `contracts/staker/src/contract/init.rs:120-200`, using a value denominated in the base denom that the same instantiation writes, rather than relying on the pre-v5 constant. We further recommend updating the comment in `contracts/staker/src/contract/exec.rs:1809` so that it does not name a fixed uzig amount, and considering an administrative message to update the tolerance, so that a misconfigured value can be corrected without a migration.

Status: Resolved

5. The migration rewrites every outstanding withdrawal record in one non-resumable transaction cost scaling with the redemption backlog

Severity: Minor

- [contracts/staker/src/contract.rs:522-562](#)
- [contracts/ledger/src/contract.rs:129-136](#)

`apply_v5_redenomination` loads and re-saves every entry of `WITHDRAWAL_QUEUE`, `USER_WITHDRAWAL_REQUESTS`, `COMPLETED_UNBONDINGS` and `EMERGENCY_WITHDRAWAL_HISTORY` in a single transaction. The ledger does the same for `UNBONDING_REQUESTS`. There is no cursor, and `ContractMigrateMsg` carries no field through which one could be supplied.

Four of those five maps are pruned on settlement, so they are bounded by outstanding user activity rather than by lifetime volume. That is the concern. The cost of the migration is proportional to the size of the redemption backlog at the moment it runs. That backlog is largest during exactly the market conditions in which an urgent migration is most likely. Only `EMERGENCY_WITHDRAWAL_HISTORY` grows without bound.

The client measured the live state on a fork and records 31 withdrawal queue entries, 12 user request entries and 43 completed unbondings in section 13. Those counts sit comfortably within the block gas limit. No gas failure was demonstrated during this audit, and none is claimed. The issue is that the safety of a non-resumable, one-shot operation rests on a point-in-time measurement of a user-driven quantity, rather than on a bound enforced in code. Section 11.3 flags the same concern, with an instruction to "chunk/paginate if large" that the shipped code does not implement. The failure itself is not destructive. `BASE_DENOM.save` is the final write in both routines, so an out-of-gas migration reverts atomically and can be retried.

We classify this issue as minor since it has no impact at the current state size and may have one in the future.

Recommendation

We recommend making the redenomination resumable, by adding a cursor to the migration message and processing a bounded number of entries per call. Since a resumable sweep leaves the contract holding a mixture of scaled and unscaled amounts between calls, we further recommend recording the sweep's progress in storage and rejecting every state-mutating entry point until it completes, so that no operation can read a partially scaled state. This must cover the operational handlers `Sync`, `ReconcileLedgers`, `CollectUnbondedAmounts`, `DistributeRewardsAndFees` and `DistributeLosses`, not only the user-facing ones. `BASE_DENOM` should continue to be written last, so that the completion flag and the denomination flip commit together.

Status: Acknowledged

The client acknowledges the issue stating that the affected maps are bounded by open user activity rather than lifetime volume, and live state measurements (31 withdrawal queue entries, 12 user request entries, 43 completed unbondings) fall comfortably within block gas limits. The client also notes that an out-of-gas migration reverts atomically, since `BASE_DENOM.save` is the final write, allowing a clean retry. Implementing a resumable cursor would require blocking all state-mutating entry points mid-sweep, a complexity the client considers disproportionate to the current risk, with a commitment to revisit if the backlog grows significantly.

6. Both contracts store the base denomination under the same key, so recovering from a mis-targeted migration silently leaves the ledger unscaled

Severity: Minor

- [contracts/staker/src/state.rs:7](#)
- [contracts/ledger/src/state.rs:8](#)
- [contracts/staker/src/contract.rs:494](#)
- [contracts/staker/src/contract.rs:512](#)
- [contracts/ledger/src/contract.rs:104](#)

Both contracts declare their new denomination marker as `Item::new("base_denom")`, using the same key string, and each one gates its one-time redenomination on that key.

`execute_migrate_ledger` accepts any `new_code_id` without verifying that it is a ledger build, so the staker's own build can be migrated onto a ledger address. That runs the staker's `apply_v5_redenomination` against ledger storage, and it does not revert. Every access is a `may_load`, including `PROTOCOL_INFO.may_load` in `contracts/staker/src/contract.rs:512`, and the scale closure in line 494 skips absent and zero items. The routine finds nothing to scale, writes "azig" to the shared key, and commits successfully.

The mis-targeted migration is an obvious mistake, and the operator would repair it by migrating the correct ledger build. That recovery step is what makes this worth reporting. It appears to succeed, but the ledger's own routine now reads `base_denom(storage) == "azig"` in `contracts/ledger/src/contract.rs:104` and returns early. `BONDED_AMOUNT`, `UNBONDING_AMOUNT`, `UNBONDED_AMOUNT`, the reward and slash items, and every unbonding request all stay 10^{12} times too small, and no error is raised anywhere. If the two contracts used different key names, the recovery would have worked.

We classify this issue as minor since it requires the migration authority to supply an incorrect code identifier. That is a mistake rather than a routine use of the role, and the resulting state is repairable by a purpose-built contract version.

Recommendation

We recommend giving each contract its own storage key for the denomination marker, and verifying the target's contract name through `cw2::get_contract_version` before any state is modified.

Status: Resolved

7. The token registration fee floor and its documentation still express the pre-migration unit

Severity: Informational

- [contracts/staker/src/contract/exec.rs:102-128](#)

Line 125 was rewritten so that the payment denomination is read from storage. The threshold on the next line, `Uint256::from(1000u32)`, was left at the pre-migration scale. So were the documentation comments in lines 102 and 123, and the `required` string returned to the operator. This is the class of defect that section 4.1 of the migration plan set out to sweep, and it is the only instance the sweep missed. `MAX_PRECISION_LOSS` was caught because it is a stored item, whereas a denomination-string sweep cannot see a bare integer. The point is therefore about the completeness of the sweep's method rather than about this constant on its own.

The comparison is unreachable on any deployed staker, which is why it is rated at this level. `TOKEN_CONTRACT` is written exactly once and never reset, so lines 118-121 return `TokenAlreadyRegistered` before `must_pay` in line 125 is reached. The check is also redundant. The forwarded funds meet the token factory inside a `reply_on_success` submessage, so underpayment reverts the whole transaction and returns them. This finding assumes that the chain's denomination creation fee is itself rescaled at v5, which is outside the audited scope.

We classify this issue as informational since the code is unreachable in the frozen commit, which the severity criteria route to this category.

Recommendation

We recommend removing the redundant floor and relying on the chain's own enforcement. An alternative is to store the threshold as an item that the redenomination scales, as is already done for `MAX_PRECISION_LOSS`. In either case, the documentation comments should stop naming the retired denomination.

Status: Resolved

Proof of Concept

P.1 — The documented recovery procedure, run during the migration window that the pause flag fails to close, double-scales bonded_amount and settles queued withdrawals at 1% of their value

← [Back to finding_1](#)

Both tests are appended to `contracts/staker/src/multitest/redenom_test.rs` and pass. Each one runs the full runbook twice, and the two runs differ by exactly one message.

```
/// Drive the full v5 sequence, optionally landing ONE `reconcile_ledgers` in
the
/// window between the chain upgrade and the ledger migrate.
fn v5_sequence(reconcile_in_window: bool) -> (Uint256, Uint256) {
    let mut f = setup();

    // A real position: 100 ZIG deposited and bonded to the validator.
    let user = api_zig().addr_make("poc_user");
    fund(&mut f.app, &user, "uzig", 100_000_000);
    f.staker.deposit(&mut f.app, &user.to_string(),
        vec![Coin { denom: "uzig".to_string(), amount: Uint256::new(100_000_000)
    }]).unwrap();
    f.staker.sync(&mut f.app, &f.admin).unwrap();
    let funds_before =
f.staker.query_funds_raised(&f.app).unwrap().funds_raised;

    // ZIGChain v5 upgrade at height H: bank and x/staking rescale x1e12, uzig -
> azig.
    f.app.simulate_v5 redenomination();

    // THE WINDOW. `ReconcileLedgers` carries no PAUSED gate, and it is step 1
of
    // the documented Recovery Procedure in contracts/staker/README.md:315-320.
    if reconcile_in_window {
        f.staker.reconcile_ledgers(&mut f.app, &f.admin)
            .expect("reconcile must succeed in the window: that is the whole
problem");
    }

    // Runbook step 4: migrate the staker, then every ledger. Two transactions.
    let staker_code_id = StZIGStakerWrapperContract::store_code(&mut f.app);
    f.staker.migrate(&mut f.app, &f.admin, staker_code_id,
        &crate::msg::ContractMigrateMsg { new_admin: None }).unwrap();
    let ledger_code_id = StZIGStakerWrapperContract::store_ledger_code(&mut
f.app);
    f.staker.migrate_ledger(&mut f.app, &f.admin, ledger_code_id).unwrap();

    // Steps 5 and 6: unpause, the operator resumes. Reconciliation publishes
the
    // slash and distribute_losses applies it to FUNDS_RAISED_BALANCE.
```

```

f.staker.reconcile_ledgers(&mut f.app, &f.admin).unwrap();

// Attribution: funds_raised is still correct here, so whatever changes it
below
// is distribute_losses and nothing else. Both migrations have already run.
let funds_before_losses =
f.staker.query_funds_raised(&f.app).unwrap().funds_raised;
assert_eq!(funds_before_losses, funds_before * Uint256::new(V5_SCALE),
    "post-migration, pre-distribute_losses: funds_raised is exactly x1e12 in
    BOTH arms");

// Must SUCCEED, not be swallowed, or the write-down cannot be attributed to
it.
f.staker.distribute_losses(&mut f.app, &f.admin)
    .expect("distribute_losses must succeed; a swallowed error would break
    attribution");

(funds_before, f.staker.query_funds_raised(&f.app).unwrap().funds_raised)
}

#[test]
fn poc_oak_1_window_reconcile_double_scales_and_destroys_funds_raised() {
    let scale = Uint256::new(V5_SCALE);

    // BASELINE, executed rather than asserted in a comment: the project's own
    // ordering, with no reconciliation in the window. This is correct behavior.
    let (base_before, base_after) = v5_sequence(false);
    assert_eq!(base_after, base_before * scale,
        "BASELINE: with no reconcile in the window, funds_raised scales exactly
        x1e12");

    // EXPLOIT: byte-identical sequence plus ONE `reconcile_ledgers` in the
    window.
    let (poc_before, poc_after) = v5_sequence(true);
    let correct = poc_before * scale;
    assert!(poc_after < correct / Uint256::new(1000), "EXPLOIT: funds_raised
    must collapse");

    // The write-down is bounded by the basis-point floor at 9999/10000, not
    total.
    assert_eq!(poc_after, correct - (correct * Uint256::new(9999) /
    Uint256::new(10000)),
        "the loss is 99.99%, capped by FEE_PERCENTAGE_DENOMINATOR, leaving
        exactly 1/10000");
}

/// Second impact: a routine `sync` in the same window zeroes the buffers
without
/// bonding anything. `v5_buffer_sequence` deposits without bonding, optionally
/// syncs in the window, then migrates and syncs again.
#[test]
fn poc_oak_1b_window_sync_zeroes_buffers_without_bonding() {
    let scale = Uint256::new(V5_SCALE);

```

```

let d = Uint256::new(100_000_000u128);

// BASELINE: no sync in the window. The buffer survives, is scaled by the
// migration, and the first post-migration sync bonds it.
let (base_buffer, _base_balance, base_bonded) = v5_buffer_sequence(false);
assert_eq!(base_buffer, Uint256::zero(), "BASELINE: the buffer is consumed
by bonding");
assert_eq!(base_bonded, d * scale, "BASELINE: the whole deposit ends up
delegated");

// EXPLOIT: identical, plus one `sync` in the window.
let (poc_buffer, poc_balance, poc_bonded) = v5_buffer_sequence(true);
assert_eq!(poc_buffer, Uint256::zero(), "the buffer is zeroed by erasure,
not by bonding");
assert_eq!(poc_bonded, Uint256::zero(), "EXPLOIT: nothing was ever
delegated");
assert_eq!(poc_balance, d * scale,
    "EXPLOIT: the funds are still physically held by the contract, now
unattributed");
}

```